Webhooks & Client API

Client API provides the opportunity for its users to be proactively informed about some changes without the need to poll requests for each update with the according API endpoint. Currently, the following three event types are available for all customers to freely subscribe to through webhooks.

- Order Status Updates
- Print Data Status Updates
- Print Data Check Status Updates
- Tracking Links Updates

In order to activate the webhook feature, the Client API users should only provide the URL (to which they want to receive the events) to FLYERALARM, while requesting the demo and/or live tokens for its inclusion in the creation process.

Order Status Updates

All Client API users are welcome to subscribe to this event, allowing them to get updates on the current status of the placed order.

The possible order status updates mentioned below are available and correspond to those listed in the response returned by the API endpoint: **/{country}/v2/orders/{order_id}/status_info**.

- Order Accepted
- Printing data accepted
- In Production
- Shipping
- Printing Data Missing
- Printing Data Invalid
- Print Data Uploaded
- Data Check in Progress
- Contact Support
- Print Approval Missing
- Print Approval Given
- Orden Canceled
- Refunded

The sample below shows the structure of the event payload for order status updates.


```
{
  "type": "OrderStatusV1",
  "submitted_at": "2025-01-01T00:00:00+02:00",
  "order_id": "DE123456789",
  "order_status_id": 10,
  "order_status": "Order Accepted"
```

}

Print Data Status Updates

All Client API users are welcome to subscribe to this event, allowing them to get updates on the current status of the print data already uploaded for the placed order.

The possible print data status updates mentioned below are available and equivalent to the response returned by the API endpoint: **/{country}/v2/orders/{order_id}/print_data/status.**

- Processing
- Print Data Check Accepted
- Print Data Check Rejected

The sample below shows the structure of the event payload for print data status updates.


```
{
  "type": "PrintDataStatusV1",
  "submitted_at": "2025-01-01T00:00:00+02:00",
  "order_id": "DE123456789",
  "order_upload_status_id": 0,
  "order_upload_status": "Processing",
  "reasons": [
    "Page1: [[ reason for rejection ]]",
    "Page2: [[ reason for rejection ]]",
    "Page99: [[ reason for rejection ]]"
  ]
}
```


Print Data Check Status Updates

This webhook can be used for print data file check executed from Variant ID. All Client API users are welcome to subscribe to this event, allowing them to get updates on the current status of the print data check based on the variant ID before the data upload.

The possible print data status updates mentioned below are available and equivalent to the response returned by the API endpoint: **/{country}/v2/orders/print_data_checks/{print_data_check_id}.**

- Processing
- Print Data Check Accepted
- Print Data Check Rejected

The sample below shows the structure of the event payload for order status updates.


```
{
  "type": "CheckPrintDataStatusV1",
  "submitted_at": "2025-01-01T00:00:00+02:00",
  "data_check_id": "B3KEnO99gx68",
  "order_upload_status_id": 0,
  "order_upload_status": "Processing",
  "reasons": [
    "Page1: [[ reason for rejection ]]",
    "Page2: [[ reason for rejection ]]",
    "Page99: [[ reason for rejection ]]"
  ]
}
```


Tracking Links Updates

All Client API users are welcome to subscribe to this event, allowing them to get the tracking link of the placed order as soon as it is available in the system.

The sample below shows the structure of the event payload for tracking link updates.


```
{
  "type": "TrackingLinksV1",
  "submitted_at": "2025-01-01T00:00:00+02:00",
  "order_id": "DE123456789",
  "carrier": "UPS",
  "shipment_data": [
    {
      "id": "Parcel1",
      "url": "https://ups.com/?loc=de_DE&ref=Parcel1",
      "carrier": "UPS",
      "created": null
    }
  ]
}
```

In addition to already described events, Client API also provides a “dummy webhook”, allowing its users to test their webhook configurations and the provided URL for extra security. This way they can test the webhooks and receive sample events before the real orders placement: **/{country}/v2/webhooks/test**.

Webhook Management

All users are welcome to include up to **10** webhooks per one JWT token. The interval between the failed attempts is **5 seconds**. The users have the opportunity of **two** attempts, excluding the initial try.

If the token is renewed due to its expiration date, the new token will inherit the duplicated set of all webhooks from the previous JWT token.

The webhook body will be a JSON resource object that relates to the event. The request headers will include:

- A Content-Type header set to **application/json**
- A **X-webhook-unixtimestamp** with the time the webhook was sent
- A **x-webhook-signature** header containing the request signature. Read on for information about validating request signatures.

Failure conditions

Flyeralarm considers the following scenarios as failure conditions:

- **Non-200 HTTP status code response:** If we receive any response other than a valid HTTP 200 response, it is regarded as a failure.
- **Redirects:** If the webhook encounters redirects while attempting to deliver the payload, it will be treated as a failure.
- **SSL Certificate Issues:** If we cannot successfully negotiate or validate your server’s SSL certificate, it will be marked as a failure.

Validating request signatures

To ensure the authenticity of a webhook request, validate the request signature using the provided headers and your webhook’s associated signing key.

- Check the **x-webhook-unixtimestamp** header to ensure the request is recent. A request older than 5 minutes may indicate a replay attack.
- Retrieve the timestamp from the **x-webhook-unixtimestamp** header.
- Concatenate the timestamp and the request body with a colon (:) separator. timestamp + ":" + JSON.stringify(request_body)
- Use your webhook’s associated signing key and the SHA-256 hashing algorithm to generate the HMAC hash.

- Compare **x-webhook-signature** and the generated HMAC hash. If they match, the request is authentic.

Code example:

```
$signingKey = "signing-key-taken-from-the-webhook-endpoint";
$request_body = [
    "type" => "DummyV1",
    "submitted_at" => "2025-09-16T14:40:21+02:00",
    "message" => "Dummy message"
];
$bodyHash = hash_hmac('sha256', $timestamp . ':' . json_encode($request_body, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR), $signingKey);
if ($bodyHash === $request->headers['x-webhook-signature']) {
    echo 'All good';
}
```

The Client API users subscribed to any of the webhooks provided have also the opportunity to manage them with the following functions:

- Review the lists of the available webhook URLs included in the certain token
- Add a new webhook URL to the certain token
- Deactivate the previously included webhook URL from the list
- Reactivate the previously deactivated webhook URL from the list
- Send a test for a specific webhook type

Feel free to contact us via service@flyeralarm.pro in case of any questions.